Java Data Structures

> → **Key Idea**: Master Java's built-in data structures for efficient interview problem solving

> • **When to use**: Interview implementation, understanding performance characteristics, choosing right collections

> ⊕ **Complexity**: Varies by data structure, HashMap O(1), TreeMap O(log n), ArrayList O(1) amortized

> ⚠ **Common Pitfall**: Using wrong collection type or not understanding performance implications

Overview

Java Collections Framework provides efficient implementations of common data structures. Choosing the right one depends on operation requirements and performance characteristics.

Visual Example

Java Collections Hierarchy

```
                Collection
               /     |     \
          List     Set     Queue
         /   \      |       |
ArrayList  LinkedList  HashSet  PriorityQueue
                |       |
            TreeSet  LinkedHashSet

              Map
             /   |   \
    HashMap TreeMap LinkedHashMap
```

Data Structure Selection Guide

```
Need fast lookup by key?
├─ Yes → HashMap (O(1)) or TreeMap (O(log n) + sorted)
└─ No → Continue...

Need ordered elements?
├─ Yes, by insertion → LinkedHashMap / LinkedHashSet
├─ Yes, sorted → TreeMap / TreeSet
```

```
└─ No → HashMap / HashSet

Need fast access by index?
├─ Yes → ArrayList (O(1) access)
└─ No, frequent insert/delete → LinkedList (O(1) at ends)

Need FIFO queue?
├─ Yes → LinkedList or ArrayDeque
└─ Need priority? → PriorityQueue (min-heap)
```

Complexity Comparison

```
             | Add     | Remove  | Get/Contains | Iteration
-------------|---------|---------|--------------|----------
ArrayList    | O(1)*   | O(n)    | O(1)/O(n)    | O(n)
LinkedList   | O(1)    | O(1)    | O(n)         | O(n)
HashMap      | O(1)    | O(1)    | O(1)         | O(n)
TreeMap      | O(log n)| O(log n)| O(log n)     | O(n)
HashSet      | O(1)    | O(1)    | O(1)         | O(n)
TreeSet      | O(log n)| O(log n)| O(log n)     | O(n)
PriorityQueue| O(log n)| O(log n)| O(1) peek    | O(n)

* Amortized - occasional resize is O(n)
```

Core Implementation

Common Collection Operations

```java
// HashMap - Key-Value storage
Map<String, Integer> map = new HashMap<>();
map.put("key", 1);
map.getOrDefault("missing", 0);  // Returns 0 if key not found
map.computeIfAbsent("key", k -> new ArrayList<>());

// HashSet - Unique elements
Set<Integer> set = new HashSet<>();
set.add(1);
set.contains(1);  // O(1)

// ArrayList - Dynamic array
List<Integer> list = new ArrayList<>();
list.add(1);
list.get(0);      // O(1)
Collections.sort(list);  // O(n log n)

// PriorityQueue - Min-heap by default
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
// Or: new PriorityQueue<>((a, b) -> b - a);
```

```java
// TreeMap - Sorted by key
TreeMap<Integer, String> treeMap = new TreeMap<>();
treeMap.firstKey();     // Smallest key
treeMap.floorKey(5);    // Largest key ≤ 5
treeMap.ceilingKey(5);  // Smallest key ≥ 5

// Deque - Double-ended queue
Deque<Integer> deque = new ArrayDeque<>();
deque.offerFirst(1);    // Add to front
deque.offerLast(2);     // Add to back
deque.pollFirst();      // Remove from front
deque.pollLast();       // Remove from back
```

Useful Patterns

```java
// Frequency count
Map<Character, Integer> freq = new HashMap<>();
for (char c : str.toCharArray()) {
    freq.merge(c, 1, Integer::sum);
    // Or: freq.put(c, freq.getOrDefault(c, 0) + 1);
}

// Graph adjacency list
Map<Integer, List<Integer>> graph = new HashMap<>();
graph.computeIfAbsent(node, k -> new ArrayList<>()).add(neighbor);

// Sliding window with deque (monotonic)
Deque<Integer> window = new ArrayDeque<>();

// Two-sum pattern with HashMap
Map<Integer, Integer> seen = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
    int complement = target - nums[i];
    if (seen.containsKey(complement)) {
        return new int[]{seen.get(complement), i};
    }
    seen.put(nums[i], i);
}
```

Time & Space Complexity

Data Structure	Add	Remove	Search	Space
ArrayList	O(1)*	O(n)	O(n)	O(n)
LinkedList	O(1)	O(1)	O(n)	O(n)

HashMap	O(1)*	O(1)*	O(1)*	O(n)
TreeMap	O(log n)	O(log n)	O(log n)	O(n)
HashSet	O(1)*	O(1)*	O(1)*	O(n)
PriorityQueue	O(log n)	O(log n)	O(n)	O(n)

*Amortized average case

← Previous: Math & Bit Manipulation | Back to Main Guide | Next: Flow Algorithms →